PW-RES-002 · DOCUMENTO DE INGENIERIA

PitWall — Investigación técnica

Fuentes, especificaciones, ecosistema y comparativas previas al diseño

VERSION DEL DOCUMENTO
1.1

ESTADO
Cerrado

FECHA
2026-09-11

AUTOR
Luis Miguel

Documento generado automaticamente a partir de `02-RESEARCH.md`. El fichero Markdown es la unica fuente de verdad; este PDF es un artefacto derivado y no debe editarse por separado.

PitWall es un proyecto no oficial sin afiliacion con Electronic Arts Inc., Codemasters, la FIA ni Formula One World Championship Limited. Las marcas citadas pertenecen a sus respectivos titulares y se emplean unicamente de forma descriptiva.

Indice

1. Objetivo y método

1.1. Prioridad de fuentes aplicada

1.2. Procedimiento seguido

2. Aclaración de alcance: el juego, no la Fórmula 1 real

3. La especificación oficial

3.1. Inventario de adjuntos del hilo oficial

3.2. Versión adoptada

3.3. Discrepancia entre fuentes oficiales

3.4. Changelog oficial 2025 → 2026

3.5. Changelog oficial 2024 → 2025

3.6. Configuración y comportamiento operativo

3.7. Lo que la especificación no dice

4. Ecosistema de implementaciones

4.1. Criterios de evaluación

4.2. C# / .NET

4.3. TypeScript / Node.js

4.4. Python

4.5. Rust

4.6. C / C++

4.7. Aplicaciones de referencia (no librerías)

4.8. Conclusión sobre reutilización

5. Problemas prácticos reportados por la comunidad

6. Comparativa de stacks tecnológicos

6.1. Alternativas evaluadas para el núcleo

6.2. Decisión

6.3. Comparativa de almacenamiento

6.3.1. Comparativa de anfitrión de la interfaz

6.4. Comparativa de gráficas para el frontend

7. Conclusiones de la investigación

Sources

Official

Libraries

Reference implementations

Community / Issues

Investigación técnica

1. Objetivo y método

Este documento registra **qué se buscó, dónde, qué se encontró y qué se descartó**, para que cualquier afirmación del resto de la documentación pueda auditarse sin repetir la investigación.

1.1. Prioridad de fuentes aplicada

1. Documentación oficial de EA / Codemasters.
2. EA Forums / F1 Game Info Hub.
3. Documentos técnicos oficiales adjuntos (PDF / TXT).
4. Repositorios open source que implementen el protocolo.
5. Paquetes de npm / PyPI / NuGet / crates.io.
6. Issues y discusiones técnicas.
7. Reddit y foros **sólo** para descubrir problemas prácticos, nunca como fuente de una especificación.

1.2. Procedimiento seguido

Paso	Acción	Resultado
1	Localizar el hilo oficial de la especificación	`forums.ea.com/blog/f1-games-game-info-hub-en/…/12187347`
2	Enumerar los adjuntos del hilo por índice, en vez de fiarse de enlaces de terceros	5 adjuntos vivos: índices 4, 5, 8, 9, 10
3	Descargar los 5 y leer su `Content-Disposition` para obtener el nombre real	Nombres canónicos confirmados
4	Convertir los PDF a texto con conservación de layout	Texto auditable
5	**Ejecutar un diff literal** entre los `.txt` de structs 2025 y 2026	Lista de cambios objetiva, no reconstruida de memoria
6	Contrastar el `.txt` 2026 contra el PDF 2026 v1.2	**Discrepancia detectada** (§3.3)
7	Revisar el ecosistema de librerías por lenguaje	§4
8	Buscar problemas prácticos reportados	§5

> **Nota metodológica.** El paso 5 es el que da valor real a esta investigación. Enumerar «lo que cambió en 2026» de memoria o desde artículos habría producido una lista plausible pero incompleta —y probablemente habría omitido la cuantización de las g-forces o el cambio de `m_engineTemperature`, que son exactamente los que rompen un decoder en silencio—.

---

2. Aclaración de alcance: el juego, no la Fórmula 1 real

Se descarta explícitamente como fuente primaria:

Fuente	Naturaleza	Veredicto
FastF1	Datos de la F1 real vía la API de cronometraje	**Fuera de alcance.** No tiene relación con la salida del videojuego.
OpenF1	API pública de datos de la F1 real	**Fuera de alcance.**
Formula 1 Live Timing	Cronometraje oficial de la competición real	**Fuera de alcance.**

La cadena de datos de este proyecto es, y sólo es:

```
EA SPORTS F1 25 / 2026 Season Pack  →  salida UDP  →  PitWall
```

Un uso futuro como **fuente secundaria opcional** (por ejemplo, contexto de un circuito real para compararlo con el del juego) queda registrado como RF-901, desactivado por defecto y claramente separado.

---

3. La especificación oficial

3.1. Inventario de adjuntos del hilo oficial

Índice	Nombre real (según `Content-Disposition`)	Tipo	Tamaño	Rol
4	`Data Output from F1 25 v3.pdf`	PDF	659 792 B	Especificación 2025
5	`F1 25 Telemetry Output Structures.txt`	TXT	51 656 B	Structs 2025
8	`2026 Season Pack Telemetry Output Structures (1).txt`	TXT	55 368 B	Structs 2026
9	`Data Output from F1 25 2026 Season Pack (1).pdf`	PDF	752 524 B	Especificación 2026, revisión anterior
10	`Data Output from F1 25 2026 Season Pack (Season 8).pdf`	PDF	1 072 296 B	**Especificación 2026 v1.2 — fuente de verdad**

Los índices 1, 2, 3, 6, 7 y ≥ 11 devuelven 404: no existen adjuntos más recientes que el 10 en el momento de la consulta.

3.2. Versión adoptada

`Data Output from F1® 25: 2026 Season Pack` — Version 1.2.

Nota de revisión textual del documento: *«Added F2 Driver IDs as part of the Season 8 "Sports Update"»*.

Comparación entre las dos revisiones 2026 disponibles:

Aspecto	Revisión anterior (adjunto 9)	Version 1.2 (adjunto 10)
Structs	Idénticos	Idénticos
Tamaños de paquete	Idénticos	Idénticos
Frecuencias	Idénticas	Idénticas
Apéndice de Driver IDs	Menor	**Ampliado con pilotos de F2**

Conclusión operativa: las revisiones de EA dentro de un mismo formato han sido, hasta ahora, **de contenido y no de layout**. Es una señal favorable para la estrategia de decoders, pero no una garantía: el diseño asume lo contrario (RSK-01).

3.3. Discrepancia entre fuentes oficiales

Comparando el `.txt` 2026 con el PDF v1.2 aparecen elementos que **sólo están en el PDF**:

```
// Presente en el PDF v1.2, ausente en el .txt oficial de structs
struct
{
    uint8 reason;   // 0 = Wet track, 1 = Safety car deployed, 2 = Red flag
} PartialModeEnabled;
```

Y cuatro códigos de evento:

Código	Significado
`"PMEN"`	Race control have enabled Partial mode
`"PMDI"`	Race control have disabled Partial mode
`"OVEN"`	Race control have enabled Overtake mode
`"OVDI"`	Race control have disabled Overtake mode

Regla adoptada (RN-015): ante conflicto entre fuentes oficiales, prevalece el **PDF de mayor versión**. El `.txt` se usa como comodidad de transcripción, nunca como autoridad. El árbitro final, cuando exista, es un fixture capturado del juego real.

Esta discrepancia es además una lección de método: **una única fuente oficial no basta cuando existen varias**. El decoder implementa los 25 códigos del PDF, no los 21 del `.txt`.

3.4. Changelog oficial 2025 → 2026

Transcrito del PDF v1.2, con el impacto anotado:

Cambio declarado por EA	Impacto en PitWall
«[IMPORTANT] Increasing the maximum number of vehicles to 24… so most packet sizes have changed»	Estructural. Obliga a decoders separados.
«Engine temperature has been changed to a uint8»	Rompe el decoder en silencio si se ignora.
«G-force values have been quantised to reduce space»	Requiere desescalado ÷ 1000,0.
«Added new tracks, teams, drivers and cars for 2026 season»	Catálogos por formato.
«Increased the size of Driver, Network and Team Ids»	`uint8` → `uint16`; centinela 255 → 65535.
«Added new Car Telemetry 2 packet»	Paquete 16 nuevo.
«Added activation zones and active aero status to session packet»	Base del mapa de zonas.
«Added ERS harvest limit per lap to car status packet»	Campo nuevo.
«Added driver reaction time for race starts»	Campo nuevo.
«Added collision severity to event»	Campo nuevo en la unión.
«Added anti-lock brakes, traction control, hi-vis and colourblind assists & rewind prompt options»	Campos de contexto.
«Race control messages for partial mode and overtake mode activation have been added to the event packet»	**Confirma que los 4 códigos del §3.3 son reales**, aunque el `.txt` no los liste.

Este último punto es determinante: el changelog oficial **corrobora** la información que sólo aparecía en el PDF. La discrepancia se resuelve por dos vías independientes.

3.5. Changelog oficial 2024 → 2025

Conservado por trazabilidad, aunque el formato 2024 esté fuera de alcance:

- Tiempo de stop-go añadido al paquete de eventos.
- Porcentaje de ampollas de neumático añadido a Car Damage.
- `chassisPitch` añadido a Motion Ex.
- Colores de librea añadidos a Participants (y reducción del nombre a 32 caracteres).
- Reducción del nombre también en Lobby Info.
- `wheelCamber` y `wheelCamberGain` añadidos a Motion Ex.
- Razón detallada de la desactivación de DRS.
- Razón de retirada en el evento `Retirement`.
- Paquete Lap Positions nuevo.
- `resultReason` en Final Classification.
- Compuesto C6 documentado.

Patrón observado a lo largo de dos revisiones anuales: EA añade campos al final de los structs y paquetes nuevos al final del enumerado. Nunca reordena. Esa regularidad es la base empírica de la arquitectura de decoders (`01-REQUIREMENTS.md` §13.4).

3.6. Configuración y comportamiento operativo

Aspecto	Hallazgo
Menú	`Game Options → Settings → UDP Telemetry Settings`. Cambios con **efecto inmediato** durante la sesión.
XML (sólo PC)	`...\Documents\My Games\<game_folder>\hardwaresettings\hardware_settings_config.xml`, etiqueta `<udp enabled broadcast ip port sendRate format yourTelemetry onlineNames />`.
Prioridad	Los cambios en el juego **sobrescriben** las ediciones manuales del XML.
Broadcast	Transmite a la subred; no requiere IP destino, sólo puerto.
Compatibilidad	F1 26 soporta **sólo los dos formatos anteriores**: 2025 y 2024.
Primer fotograma	Envía 9 paquetes de inicialización con `sessionTime = 0.000`.
Agrupación	Los paquetes «rate del menú» se envían juntos en el mismo fotograma y nunca se separan.
Orden de ruedas	RL, RR, FL, FR. Constante en toda la especificación.
Índices de vehículo	Estables durante toda la sesión.
Coches	Máximo 24 en las estructuras; comprobar `m_numActiveCars` y el `m_resultStatus` individual.

3.7. Lo que la especificación no dice

Registrado como preguntas abiertas en `01-REQUIREMENTS.md` §35:

- La lista exacta de valores seleccionables de *UDP Send Rate*.

- Los valores concretos de `m_recommendedSession` en Tyre Sets (remite a un apéndice que no los enumera).
- Si `m_overallFrameIdentifier` se reinicia entre sesiones.
- Si los eventos `PMEN` / `OVEN` son globales o por vehículo.
- El orden y la equidad del ciclado de `Session History` y `Tyre Sets` con 24 coches.

Ninguna de estas lagunas bloquea el diseño: todas se resuelven midiendo, y el sistema está construido para medir (RF-801).

---

4. Ecosistema de implementaciones

4.1. Criterios de evaluación

Para cada candidato: última actualización · versión de F1 soportada · soporte del 2026 Season Pack · actividad · issues · licencia · calidad · pruebas.

4.2. C# / .NET

Proyecto	`F1Game.UDP`
Versión	26.0.0
Publicación	2026-06-18
Objetivos	.NET 8, 9, 10
Licencia	**MIT**
Soporte 2026	**Sí**, layout del 2026 Season Pack, incluye `CarTelemetry2DataPacket`
Arquitectura	Extensión `ToPacket` que convierte un `byte[]` en un `UnionPacket`; se consulta `PacketType` y se accede con `TryGet<Nombre>`
Convención	La versión mayor de la librería sigue el año del juego (26.x → F1 26)
Valor para nosotros	**La mejor referencia disponible.** Confirma que el mapeo directo `bytes → struct` es la forma natural de resolver este protocolo en un lenguaje con control de layout.

Es la única implementación verificada con soporte real y actual del formato 2026.

4.3. TypeScript / Node.js

Proyecto	Licencia	Soporte	Estado
`@racehub-io/f1-telemetry-client`	MIT	Hasta F1 23	Renombrado desde `f1-telemetry-client`. Sin 2025/2026.
`@z0mt3c/f1-telemetry-client`	—	Hasta F1 24	Bifurcación mantenida más tiempo. Sin 2026.
`@deltazeroproduction/f1-udp-parser`	—	Hasta F1 25	El más actual del ecosistema JS. **Sin 2026.**

Conclusión: el ecosistema TypeScript va por detrás. Ninguna opción cubre el formato 2026.

4.4. Python

Proyecto	Estado
`f1-telemetry` (PyPI, publicado 2026-06-05)	Aplicación con InfluxDB para telemetría en vivo. Referencia de arquitectura con base de datos de series temporales, no de decoder.
Diversos parsers en GitHub	Mayoritariamente basados en `struct.unpack` con cadenas de formato. Cobertura irregular de 2025/2026.

Observación de valor: el uso de InfluxDB en estos proyectos es una decisión que aquí **se descarta** conscientemente (`15-ADR.md`, ADR-005): un servidor de series temporales es infraestructura desproporcionada para una aplicación de un único usuario en un único PC.

4.5. Rust

Proyecto	Estado
`mathieu-lemay/f1-telemetry-rs`	Librería de parseo con dashboard. Soporte de 2026 no verificado.
`f1-game-library-models-25` (crates.io)	Modelos para el formato 2025.

Referencia útil de separación decoder/dominio en un lenguaje sin recolector de basura, que es exactamente nuestro caso.

4.6. C / C++

Proyecto	Estado
`MacManley/f1-25-udp`	Parser para ESP32/ESP8266, formato 2025.

El hueco identificado: no existe una librería de C++ moderna, mantenida y con soporte del formato 2026. Éste es el argumento decisivo del ADR-004: no se rechaza una librería madura, es que **no la hay** para el stack elegido.

4.7. Aplicaciones de referencia (no librerías)

Se han observado como referencia de producto, no de código:

Categoría	Qué aporta
Dashboards y overlays comerciales	Confirman qué información se consulta de reojo durante la conducción y cuál sólo tiene sentido después.
Herramientas de análisis de vueltas	Confirman que la comparación **por distancia** es el estándar del sector; comparar por tiempo es un error de principiante.
Grabadores de sesión comerciales	Confirman que la captura cruda es la práctica habitual y que la reproducción es una funcionalidad esperada, no exótica.
Software de retransmisión	Confirma la utilidad del modo broadcast y del acceso desde varios dispositivos.

4.8. Conclusión sobre reutilización

Opción	Veredicto
Adoptar una librería de decoder	**Descartada.** No existe para C++ con soporte 2026. Portar la de C# no aporta sobre transcribir la especificación.
Copiar código de referencia	**Descartada.** Aunque hay licencias MIT compatibles, la transcripción desde la especificación oficial es más corta, más trazable y más segura.
Usar librerías de infraestructura (red, logs, compresión, pruebas, gráficas)	**Adoptada.** Ahí sí se reutiliza, con licencias permisivas verificadas.

5. Problemas prácticos reportados por la comunidad

Fuentes de nivel 6 y 7. Usadas **exclusivamente** para descubrir riesgos, nunca para afirmar nada sobre el protocolo.

Problema	Lo que enseña	Requisito que lo cubre
Herramientas de terceros rotas tras el lanzamiento del 2026 Season Pack	El cambio de formato rompe integraciones que asumen un layout fijo	RF-101, RF-105, RF-800
Lecturas de telemetría incorrectas en periféricos con el 2026 SP	No dar por hecho que el formato configurado es el que llega	RF-801, pantalla *Diagnostics*
«Datos desconocidos» por otra aplicación escuchando en el mismo puerto	Un puerto UDP no es exclusivo; puede haber varios emisores y varios receptores	RF-009, RF-803
Confusión entre el formato del juego base y el del DLC	El usuario puede tener el formato 2025 seleccionado sin saberlo, con datos degradados	RF-104
Pérdida de datos a 60 Hz en redes con carga	El send rate alto no siempre es la mejor opción	RF-802, medición de pérdida

6. Comparativa de stacks tecnológicos

6.1. Alternativas evaluadas para el núcleo

Criterio	C++20/23	Rust	C# / .NET	TypeScript / Node	Python
Mapeo directo de structs `packed`	**Excelente** (`#pragma pack`, `static_assert`)	Excelente (`repr(C)`, zerocopy)	Muy bueno (`StructLayout`, `MemoryMarshal`)	Malo (lectura por offsets a mano)	Malo (`struct.unpack`)
Verificación de tamaños en compilación	**Sí**	Sí	Parcial	No	No
Latencia y jitter	**Excelente**	Excelente	Muy bueno	Bueno (con cuidado de GC)	Aceptable
Control explícito de concurrencia	**Total**	Total con garantías	Alto	Limitado	Limitado por el GIL
Coste de aprendizaje	Alto	Alto	Bajo	Bajo	Muy bajo
Ecosistema de librerías del dominio	Escaso	Escaso	**El mejor** (`[S6]`)	Medio	Medio
Distribución en Windows	Ejecutable nativo	Ejecutable nativo	Requiere runtime	Requiere Node	Empaquetado incómodo
Riesgo de errores de memoria	**Alto** (mitigable)	Muy bajo	Muy bajo	Muy bajo	Muy bajo
Ajuste al objetivo formativo declarado	**Máximo**	Alto	Medio	Bajo	Bajo

6.2. Decisión

Núcleo en C++20/23; interfaz en React + TypeScript.

El razonamiento honesto, incluidas las contrapartidas:

- **A favor.** El problema es un protocolo binario `packed` de tamaño fijo con requisitos de latencia y concurrencia explícita. C++ permite transcribir la especificación de forma literal y que **el compilador verifique** que el layout coincide (`static_assert`). Ninguna otra opción del listado ofrece a la vez esa verificación estática y el control total del modelo de hilos. Además, el objetivo formativo declarado —C++ moderno, concurrencia, redes, optimización y sistemas de baja latencia— es un requisito del proyecto, no un capricho: un stack que lo ignore fallaría en un objetivo real.
- **En contra, y asumido.** C++ es la opción con mayor riesgo de errores de memoria (RSK-06) y con la mayor pendiente de aprendizaje (RSK-04). Se mitiga con `std::span` en lugar de punteros, comprobación estricta de tamaños antes de cualquier lectura, fuzzing continuo y desinfectantes de dirección e hilos en integración continua. Estas mitigaciones no son opcionales: son requisitos (SEC-16, SEC-17, RNF-054).
- **Consecuencia aceptada.** El proyecto avanzará más despacio que con TypeScript o C#. Se compensa acotando el MVP con criterios objetivos (`12-MVP.md`) y con un plan de fases pedagógico (`11-IMPLEMENTATION-PLAN.md`).

6.3. Comparativa de almacenamiento

Opción	Ventajas	Inconvenientes	Veredicto
SQLite	Transaccional, sin servidor, formato estable durante décadas, licencia de dominio público	Malo para millones de filas de series temporales	**Adoptado para el catálogo**
SQLite con extensiones	Añade capacidades analíticas	Complica la distribución	Descartado
DuckDB	Analítico y columnar, embebido, lee y escribe Parquet, licencia MIT	Formato de fichero propio menos estable entre versiones; un único proceso escritor	**Adoptado como motor analítico**, no como almacén canónico
PostgreSQL	Robusto y completo	Requiere servidor. Desproporcionado para un PC de usuario	Descartado
TimescaleDB	Excelente para series temporales	Requiere PostgreSQL	Descartado
InfluxDB	Especializado en series temporales; usado por proyectos del ecosistema	Servidor aparte, gestión de retención propia, complica la instalación	**Descartado**, aunque sea la opción habitual del ecosistema
Parquet	Columnar, muy compresible, portable, legible por cualquier herramienta de análisis	No es transaccional; requiere un motor para consultarlo	**Adoptado para las series de alta frecuencia**
Formato binario propio	Máximo control y rendimiento	Nadie más puede leerlo; hay que escribir el escritor, el lector, el índice y las migraciones	**Adoptado sólo para la captura cruda** `.f1raw`, donde la fidelidad y la escritura secuencial son el requisito y la interoperabilidad no lo es

Combinación adoptada: `.f1raw` propio (crudo) + SQLite (catálogo) + Parquet (series) + DuckDB (consultas analíticas). Cada pieza hace exactamente aquello en lo que es la mejor opción, y ninguna obliga a instalar un servidor.

6.3.1. Comparativa de anfitrión de la interfaz

Añadida al revisar el enfoque de producto (ADR-002 v2). La pregunta no es «con qué construyo la interfaz» —eso ya estaba resuelto con React— sino **dónde se ejecuta**.

Opción	Peso añadido	Lenguaje anfitrión	Veredicto
WebView2	~0 MB: runtime compartido, de serie en Windows 11	**C++**	**Adoptada**
Electron	~150 MB por aplicación	JavaScript / Node	Descartada: añade un tercer runtime a un proyecto cuyo núcleo ya es C++
Tauri	~5 MB	**Rust**	Descartada: obligaría a un tercer lenguaje y a un puente C++ ↔ Rust
Navegador (enfoque anterior)	0	—	Descartada: contradice la experiencia objetivo del producto
Interfaz nativa (Qt, WinUI, Dear ImGui)	variable	C++	Descartada: obligaría a rehacer gráficas y mapa con mucho más esfuerzo por hora

El dato que decide es que **Tauri usa WebView2 por debajo en Windows**: su valor añadido es el andamiaje en Rust, que aquí no aporta nada. Usar WebView2 directamente desde C++ da lo mismo sin el tercer lenguaje.

Riesgo asumido: el WebView2 Runtime puede faltar en Windows 10 (RSK-13). Se comprueba al arrancar (RF-632) y en el instalador (RNF-095).

6.4. Comparativa de gráficas para el frontend

Criterio	Requisito
Puntos simultáneos	≥ 100 000 sin degradar
Tecnología	Canvas o WebGL. **SVG queda descartado** por número de nodos
Interacción	Cursor sincronizado entre gráficas, zoom por eje X compartido
Peso	Bajo, sin cadena de dependencias grande
Licencia	Permisiva

Las librerías de gráficas generalistas basadas en SVG (las más populares en React) **no cumplen** el primer criterio y quedan descartadas de entrada. La selección final se cierra en `15-ADR.md`.

---

7. Conclusiones de la investigación

#	Conclusión	Consecuencia en el diseño
1	La especificación oficial es completa, precisa y suficiente para implementar el protocolo íntegro sin ingeniería inversa	Decoder propio viable (ADR-004)
2	Existen **varias** fuentes oficiales y **no coinciden**	Regla de precedencia explícita (RN-015)
3	Los cambios entre formatos siguen cuatro patrones, todos absorbibles por el decoder	Arquitectura con capa de normalización (ADR-003)
4	No existe librería de C++ con soporte 2026	Implementación propia, sin dependencia externa crítica
5	El 2026 Season Pack es un DLC, no un juego aparte, y su formato se elige en un ajuste	Detección por cabecera obligatoria (RF-101)
6	Usar el formato 2025 con contenido 2026 degrada los datos	Recomendación activa al usuario (RF-104)
7	El volumen a 60 Hz es de ~1,5 GB/h	Compresión y retención desde el MVP (RF-402, RNF-030)
8	La telemetría restringida es el error más común del ecosistema	Tratada como defecto bloqueante (RN-001)
9	Session History y Tyre Sets ciclan y no sirven para tiempo real	Antigüedad visible obligatoria (RN-010)
10	Los flashbacks rompen el orden temporal aparente	`overallFrameIdentifier` como único ordenador (RN-002)
11	Ninguna de las herramientas del ecosistema estudiadas se usa desde un navegador: todas son aplicaciones de escritorio	Refuerza el enfoque de ADR-002 v2

Sources

Consultadas el **11 de septiembre de 2026**.

Official

Título	URL / Origen	Uso
`Data Output from F1® 25: 2026 Season Pack` v1.2 (Season 8)	Adjunto 10 de `forums.ea.com/blog/f1-games-game-info-hub-en/ea-sports™-f1®25-2026-season-pack-udp-specification/12187347`	Fuente de verdad del formato 2026: structs, tamaños, frecuencias, FAQ, apéndices, datos restringidos, configuración, changelog
`2026 Season Pack Telemetry Output Structures.txt`	Adjunto 8 del mismo hilo	Transcripción de structs 2026 (incompleta, §3.3)
`Data Output from F1® 25 Game` v3	Adjunto 4 del mismo hilo	Fuente de verdad del formato 2025
`F1 25 Telemetry Output Structures.txt`	Adjunto 5 del mismo hilo	Structs 2025; base del diff
Hilo *EA SPORTS™ F1®25: 2026 Season Pack UDP SPECIFICATION*	`forums.ea.com/blog/f1-games-game-info-hub-en/…/12187347`	Localización y versionado de adjuntos
Hilo *[DISCUSSION] F1® 25 UDP SPECIFICATION*	`forums.ea.com/discussions/f1-25-general-discussion-en/…/12187351`	Canal oficial de dudas

Libraries

Título	URL	Uso
`F1Game.UDP` 26.0.0 (MIT, 2026-06-18)	`nuget.org/packages/F1Game.UDP` · `github.com/volodymyr-fed/F1Game.UDP`	Única librería madura con soporte 2026; referencia de arquitectura de parseo
`@racehub-io/f1-telemetry-client` (MIT)	`github.com/racehub-io/f1-telemetry-client`	Referencia de API de eventos; sin soporte 2025/2026
`@z0mt3c/f1-telemetry-client`	`npmjs.com/package/@z0mt3c/f1-telemetry-client`	Cobertura hasta F1 24
`@deltazeroproduction/f1-udp-parser`	`npmjs.com/package/@deltazeroproduction/f1-udp-parser`	Cobertura hasta F1 25
`f1-telemetry` (PyPI, 2026-06-05)	`pypi.org/project/f1-telemetry/`	Arquitectura con series temporales

Reference implementations

Proyecto	URL	Uso
`mathieu-lemay/f1-telemetry-rs`	`github.com/mathieu-lemay/f1-telemetry-rs`	Separación decoder/dominio sin recolector de basura
`MacManley/f1-25-udp`	`github.com/MacManley/f1-25-udp`	Parseo por offsets fijos en microcontrolador
`f1-game-library-models-25`	`lib.rs/crates/f1-game-library-models-25`	Modelos del formato 2025 en Rust

Community / Issues

Referencia	URL	Uso
SF1000 telemetry not working after release of F1 26	`forums.ea.com/idea/f1-25-bug-reports-en/…/13457278`	Riesgo: integraciones rotas tras el Season Pack
Fanatec incorrect telemetry reading [2026 SP]	`forums.ea.com/idea/f1-25-bug-reports-en/…/13472195`	Riesgo: lecturas incorrectas
Guías de configuración UDP de terceros (F1Laps, Victory Race Engineer, SIMRIG, Sim Racing Telemetry)	varias	Comportamiento por defecto, degradación con formato 2025, valores de send rate en la práctica, conflicto de puerto

Fin de `PW-RES-002` v1.1